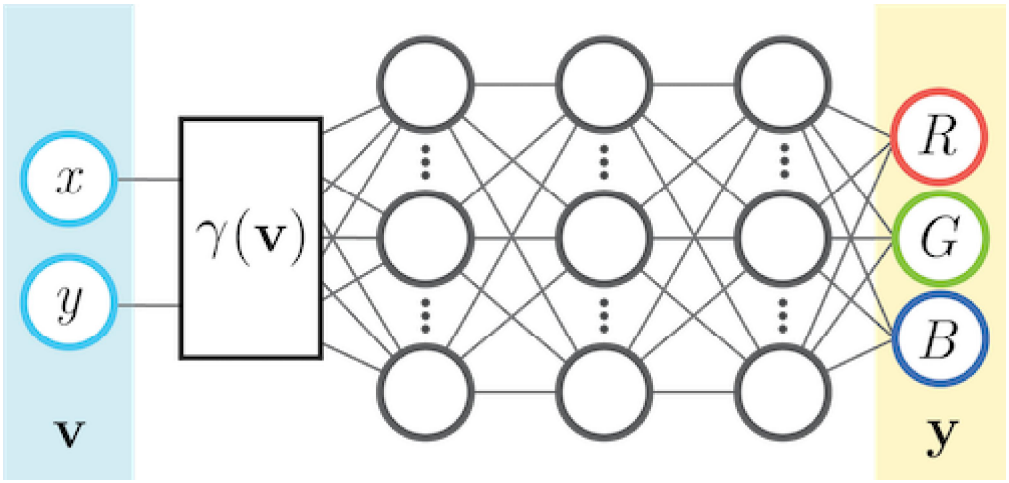
Assignment 2

In this assignment you will create a coordinate-based multilayer perceptron in numpy from scratch. For each input image coordinate (x, y), the model predicts the associated color (r, g, b).



You will then compare the following input feature mappings $\gamma(\mathbf{v})$.

- No mapping: $\gamma(\mathbf{v}) = \mathbf{v}$.
- Basic mapping: $\gamma(\mathbf{v}) = [\cos(2\pi\mathbf{v}), \sin(2\pi\mathbf{v})]^{\mathrm{T}}$.
- Gaussian Fourier feature mapping: $\gamma(\mathbf{v}) = [\cos(2\pi\mathbf{B}\mathbf{v}), \sin(2\pi\mathbf{B}\mathbf{v})]^{\mathrm{T}}$, where each entry in $\mathbf{B} \in \mathbb{R}^{m \times d}$ is sampled from $\mathcal{N}(0, \sigma^2)$.

Some notes to help you with that:

- You will implement the mappings in the helper functions `get_B_dict` and `input_mapping`.
- The basic mapping can be considered a case where $\mathbf{B} \in \mathbb{R}^{2 \times 2}$ is the indentity matrix.
- For this assignment, d is 2 because the input coordinates in two dimensions.
- You can experiment with m, like $m = 256$.
- You should show results for σ value of 1.

Source: https://bmild.github.io/fourfeat/ This assignment is inspired by and built off of the authors' demo.

Setup

(Optional) Colab Setup

If you aren't using Colab, you can delete the following code cell. Replace the path below with the path in your Google Drive to the uploaded assignment folder.

Mounting to Google Drive will allow you access the other .py files in the assignment folder and save outputs to this folder

In [ ]:
```
# you will be prompted with a window asking to grant permissions
# click connect to google drive, choose your account, and click allow
# from google.colab import drive
# drive.mount("/content/drive")
```

In [ ]:
```
# TODO: fill in the path in your Google Drive in the string below
# Note: do not escape slashes or spaces in the path string
# import os
# datadir = "/content/assignment2"
# if not os.path.exists(datadir):
#   !ln -s "/content/drive/My Drive/path/to/your/assignment2/" $datadir
# os.chdir(datadir)
# !pwd
```

Imports

In [1]:
```
import matplotlib.pyplot as plt
from tqdm.notebook import tqdm
import os, imageio
import cv2
import numpy as np

# imports /content/assignment2/models/neural_net.py if you mounted correctly
from models.neural_net import NeuralNetwork
from models.neural_net_bn import BNNeuralNetwork

# makes sure your NeuralNetwork updates as you make changes to the .py file
%load_ext autoreload
%autoreload 2

# sets default size of plots
%matplotlib inline
plt.rcParams['figure.figsize'] = (10.0, 8.0)
```

Helper Functions

Image Data and Feature Mappings (Fill in TODOs)

In [2]:
```
# Data loader - already done for you
def get_image(size=512, \
              image_url='https://bmild.github.io/fourfeat/img/lion_orig.png'):

  # Download image, take a square crop from the center
  img = imageio.imread(image_url)[..., :3] / 255.
  c = [img.shape[0]//2, img.shape[1]//2]
  r = 256
  img = img[c[0]-r:c[0]+r, c[1]-r:c[1]+r] # shape is (512, 512, 3)

  if size != 512:
    img = cv2.resize(img, (size, size))

  plt.imshow(img)
```

```
  plt.show()

  # Create input pixel coordinates in the unit square
  coords = np.linspace(0, 1, img.shape[0], endpoint=False) # shape is (size,)
  x_test = np.stack(np.meshgrid(coords, coords), axis=-1) # shape is (size, size
  test_data = [x_test, img]
  # Create a smaller training set by downsampling
  train_data = [x_test[::2, ::2], img[::2, ::2]]

  return train_data, test_data
```

In [3]:
```
# Create the mappings dictionary of matrix B -  you will implement this
def get_B_dict(size):
  # The Gaussian Fourier features mapping size is smaller than the input size
  mapping_size = size // 2 # you may tweak this hyperparameter
  B_dict = {}
  B_dict['none'] = None

  # add B matrix for basic, gauss_1.0
  # TODO implement this
  input_size = 2
  B_dict['basic'] = np.eye(input_size)
  B_dict['gauss_1.0'] = np.random.normal(0, 1, (mapping_size, input_size))

  return B_dict
```

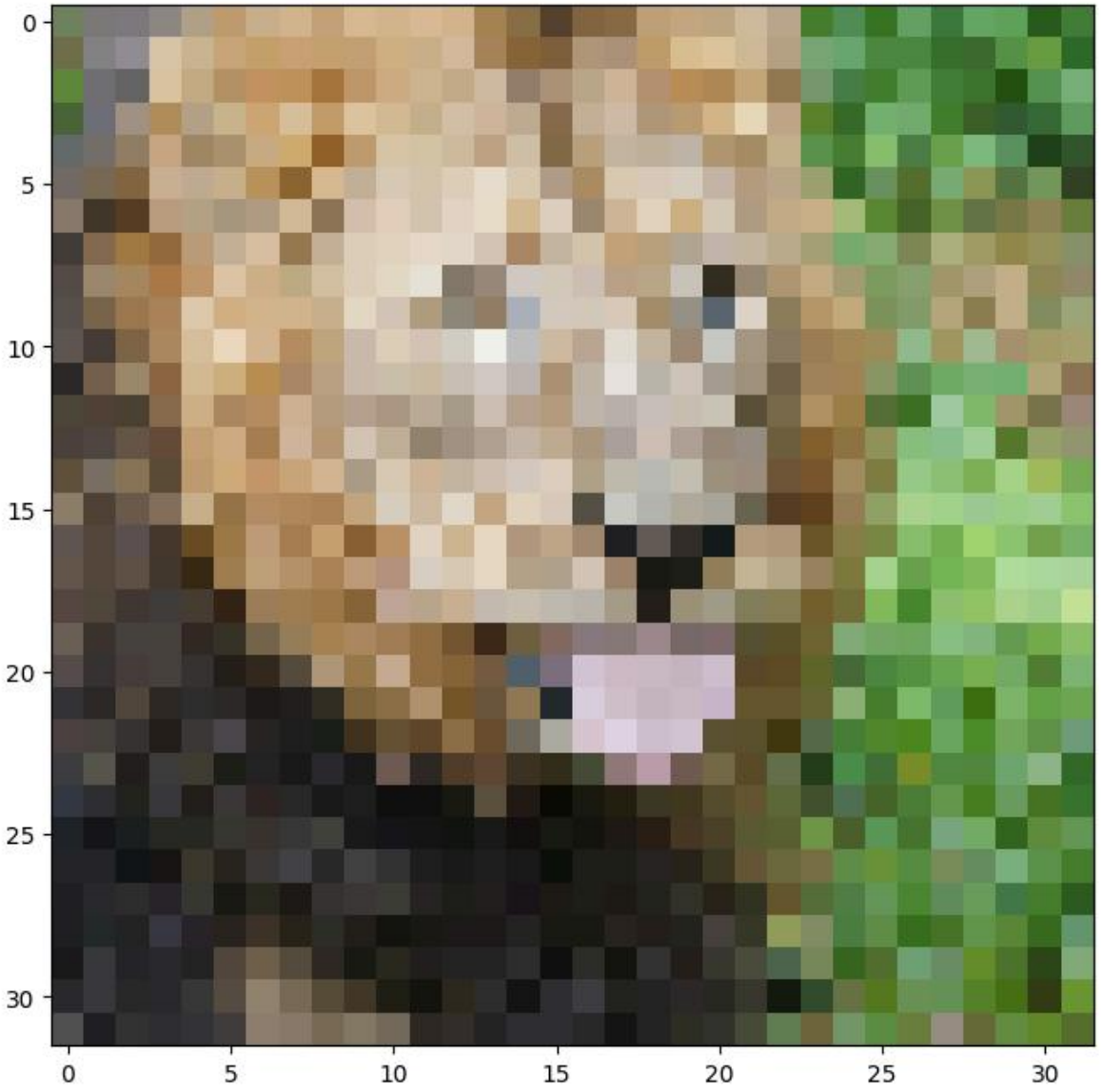
In [4]:
```
# Given tensor x of input coordinates, map it using B - you will implement
def input_mapping(x, B):
  if B is None:
    # "none" mapping - just returns the original input coordinates
    return x
  else:
    # "basic" mapping and "gauss_X" mappings project input features using B
    # TODO implement this
    N = x.shape[0]
    x_mapped = np.array([np.cos(2 * np.pi * (B @ x.T)), np.sin(2 * np.pi * (B @
    # reshape x_mapped to (N, num_features)
    x_mapped = x_mapped.reshape((N, -1))
    return x_mapped
```

MSE Loss and PSNR Error (Fill in TODOs)

In [5]:
```
def mse(y, p):
  # TODO implement this
  # make sure it is consistent with your implementation in neural_net.py
  return np.mean((y - p) ** 2)

def psnr(y, p):
  # TODO implement this
  # The maximum possible value for RGB is 255
  # return 20 * np.log10(255) - 10 * np.log10(mse(y, p))
  return -10 * np.log10(2. * mse(y, p))
```

In [6]:
```
size = 32
train_data, test_data = get_image(size)
```

```
C:\Users\wanglibin\AppData\Local\Temp\ipykernel_12200\1559754012.py:6: Deprecatio
nWarning: Starting with ImageIO v3 the behavior of this function will switch to t
hat of iio.v3.imread. To keep the current behavior (and make this warning disappe
ar) use `import imageio.v2 as imageio` or call `imageio.v2.imread` directly.
  img = imageio.imread(image_url)[..., :3] / 255.
```



Some suggested hyperparameter choices to help you start

- hidden layer count: 3
- hidden layer size: 256
- number of epochs: 1000
- learning rate: 5e-2

In [7]:
```
# TODO: Set the hyperparameters
num_layers = 4
hidden_size = 128
epochs = 3000
learning_rate = 0.01
# input_size = 2
output_size = 3
B_dict = get_B_dict(size)

print('B_dict items:')
```

```
for k,v in B_dict.items():
    print('\t',k,np.array(v).shape)
```

```
B_dict items:
        none ()
        basic (2, 2)
        gauss_1.0 (16, 2)
```

In [8]:
```
# set hyperparameters for low resolution
hidden_size_low = 512
epochs_low = 400
learning_rate_low = 0.1

# set hyperparameters for high resolution
hidden_size_high = 256
epochs_high = 3000
learning_rate_high = 0.1

# set hyperparameters for choice
hidden_size_choice = 256
epochs_choice = 3000
learning_rate_choice = 0.1

# set batch normalization
use_bn = True
```

In [9]:
```
# Apply the input feature mapping to the train and test data - already done for
def get_input_features(B_dict, mapping):
  # mapping is the key to the B_dict, which has the value of B
  # B is then used with the function `input_mapping` to map x
  y_train = train_data[1].reshape(-1, output_size)
  y_test = test_data[1].reshape(-1, output_size)
  X_train = input_mapping(train_data[0].reshape(-1, 2), B_dict[mapping])
  X_test = input_mapping(test_data[0].reshape(-1, 2), B_dict[mapping])
  return X_train, y_train, X_test, y_test
```

Plotting and video helper functions (you don't need to change anything here)

In [10]:
```
def plot_training_curves(train_loss, train_psnr, test_psnr):
  # plot the training loss
  plt.subplot(2, 1, 1)
  plt.plot(train_loss)
  plt.title('MSE history')
  plt.xlabel('Iteration')
  plt.ylabel('MSE Loss')

  # plot the training and testing psnr
  plt.subplot(2, 1, 2)
  plt.plot(train_psnr, label='train')
  plt.plot(test_psnr, label='test')
  plt.title('PSNR history')
  plt.xlabel('Iteration')
  plt.ylabel('PSNR')
  plt.legend()

  plt.tight_layout()
  plt.show()
```

```
def plot_reconstruction(p, y):
  p_im = p.reshape(size,size,3)
  y_im = y.reshape(size,size,3)

  plt.figure(figsize=(12,6))

  # plot the reconstruction of the image
  plt.subplot(1,2,1), plt.imshow(p_im), plt.title("reconstruction")

  # plot the ground truth image
  plt.subplot(1,2,2), plt.imshow(y_im), plt.title("ground truth")

  print("Final Test MSE", mse(y, p))
  print("Final Test psnr",psnr(y, p))

def plot_reconstruction_progress(predicted_images, y, N=8):
  total = len(predicted_images)
  step = total // N
  plt.figure(figsize=(24, 4))

  # plot the progress of reconstructions
  for i, j in enumerate(range(0,total, step)):
      plt.subplot(1, N+1, i+1)
      plt.imshow(predicted_images[j].reshape(size,size,3))
      plt.axis("off")
      plt.title(f"iter {j}")

  # plot ground truth image
  plt.subplot(1, N+1, N+1)
  plt.imshow(y.reshape(size,size,3))
  plt.title('GT')
  plt.axis("off")
  plt.show()

def plot_feature_mapping_comparison(outputs, gt):
  # plot reconstruction images for each mapping
  plt.figure(figsize=(24, 4))
  N = len(outputs)
  for i, k in enumerate(outputs):
      plt.subplot(1, N+1, i+1)
      plt.imshow(outputs[k]['pred_imgs'][-1].reshape(size, size, -1))
      plt.title(k)
  plt.subplot(1, N+1, N+1)
  plt.imshow(gt)
  plt.title('GT')
  plt.show()

  # plot train/test error curves for each mapping
  iters = len(outputs[k]['train_psnrs'])
  plt.figure(figsize=(16, 6))
  plt.subplot(121)
  for i, k in enumerate(outputs):
      plt.plot(range(iters), outputs[k]['train_psnrs'], label=k)
  plt.title('Train error')
  plt.ylabel('PSNR')
  plt.xlabel('Training iter')
  plt.legend()
  plt.subplot(122)
  for i, k in enumerate(outputs):
      plt.plot(range(iters), outputs[k]['test_psnrs'], label=k)
```

```
  plt.title('Test error')
  plt.ylabel('PSNR')
  plt.xlabel('Training iter')
  plt.legend()
  plt.show()

# Save out video
def create_and_visualize_video(outputs, size=size, epochs=epochs, filename='trai
  all_preds = np.concatenate([outputs[n]['pred_imgs'].reshape(epochs,size,size,3
  data8 = (255*np.clip(all_preds, 0, 1)).astype(np.uint8)
  f = os.path.join(filename)
  imageio.mimwrite(f, data8, fps=20)

  # Display video inline
  from IPython.display import HTML
  from base64 import b64encode
  mp4 = open(f, 'rb').read()
  data_url = "data:video/mp4;base64," + b64encode(mp4).decode()

  N = len(outputs)
  if N == 1:
    return HTML(f'''
    <video width=256 controls autoplay loop>
          <source src="{data_url}" type="video/mp4">
    </video>
    ''')
  else:
    return HTML(f'''
    <video width=1000 controls autoplay loop>
          <source src="{data_url}" type="video/mp4">
    </video>
    <table width="1000" cellspacing="0" cellpadding="0">
      <tr>{''.join(N*[f'<td width="{1000//len(outputs)}"></td>'])}</tr>
      <tr>{''.join(N*['<td style="text-align:center">{}</td>'])}</tr>
    </table>
    '''.format(*list(outputs.keys())))
```

Experiment Runner (Fill in TODOs)

In [11]:

```
def NN_experiment(X_train, y_train, X_test, y_test, input_size, num_layers,\
                  hidden_size, output_size, epochs, use_bn,\
                  learning_rate, opt='SGD'):

    # Initialize a new neural network model
    hidden_sizes = [hidden_size] * (num_layers - 1)

    # Build the model
    if use_bn == True:
        net = BNNeuralNetwork(input_size, hidden_sizes, output_size, num_layers,
    else:
        net = NeuralNetwork(input_size, hidden_sizes, output_size, num_layers, c

    # Variables to store performance for each epoch
    train_loss = np.zeros(epochs)
    train_psnr = np.zeros(epochs)
    test_psnr = np.zeros(epochs)
    predicted_images = np.zeros((epochs, y_test.shape[0], y_test.shape[1]))

    # For each epoch...
```

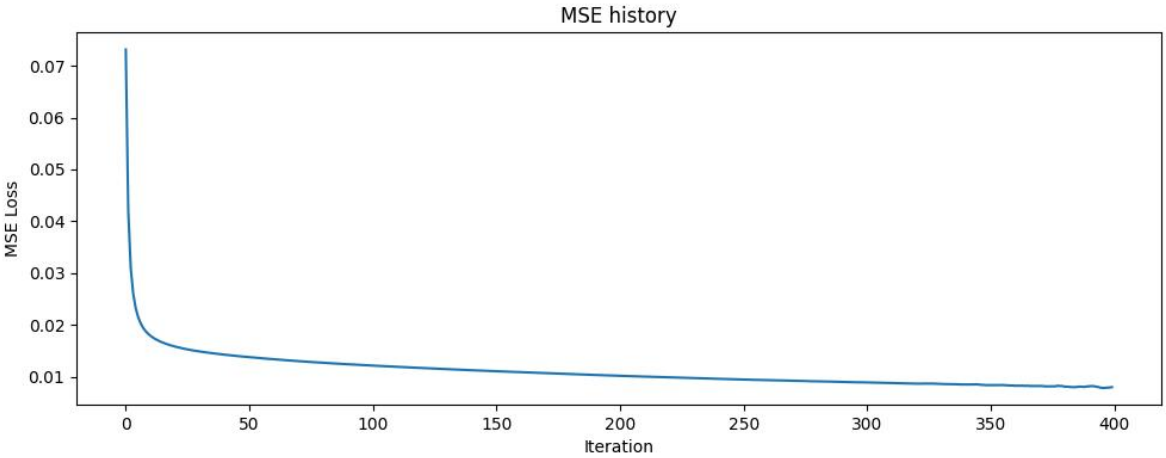
```
    for epoch in tqdm(range(epochs)):

        # Shuffle the dataset
        # TODO implement this
        # Get the number of samples in the dataset
        N = X_train.shape[0]
        # Set different random seed for each epoch
        np.random.seed(epoch)
        shuffle_indices = np.random.permutation(N)
        X_train_used = X_train[shuffle_indices]
        y_train_used = y_train[shuffle_indices]

        # Training
        # Run the forward pass of the model to get a prediction and record the p
        # TODO implement this
        pred = net.forward(X_train_used)
        # Record the psnr
        train_psnr[epoch] = psnr(pred, y_train_used)

        # Run the backward pass of the model to compute the loss, record the los
        # TODO implement this
        loss = net.backward(y_train_used)
        # Record the loss and update the weights
        train_loss[epoch] = loss
        net.update(learning_rate)

        # Testing
        # No need to run the backward pass here, just run the forward pass to co
        # TODO implement this
        test_pred = net.forward(X_test)
        # Record the psnr and predicted images
        test_psnr[epoch] = psnr(test_pred, y_test)
        predicted_images[epoch] = test_pred

    return net, train_psnr, test_psnr, train_loss, predicted_images
```

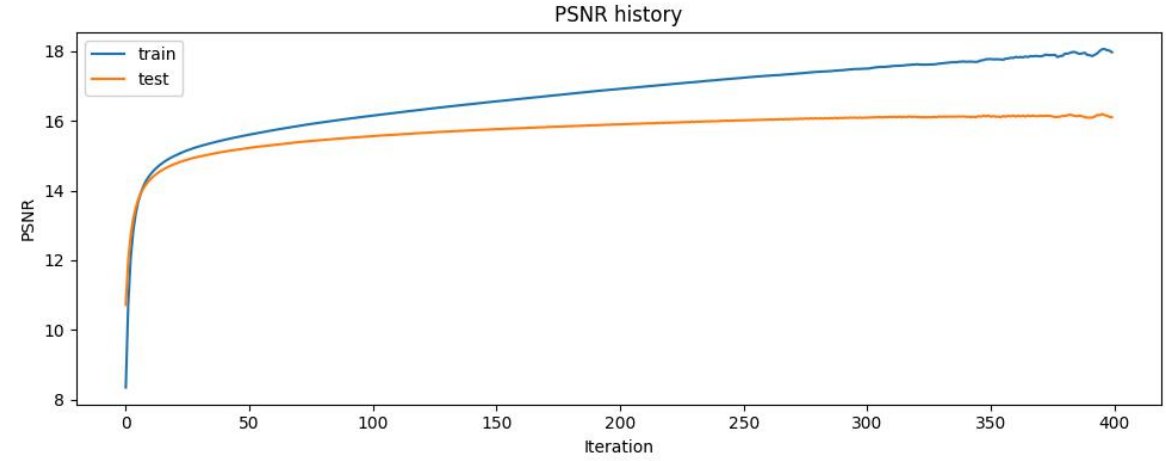
Low Resolution Reconstruction

Low Resolution Reconstruction - SGD - None Mapping

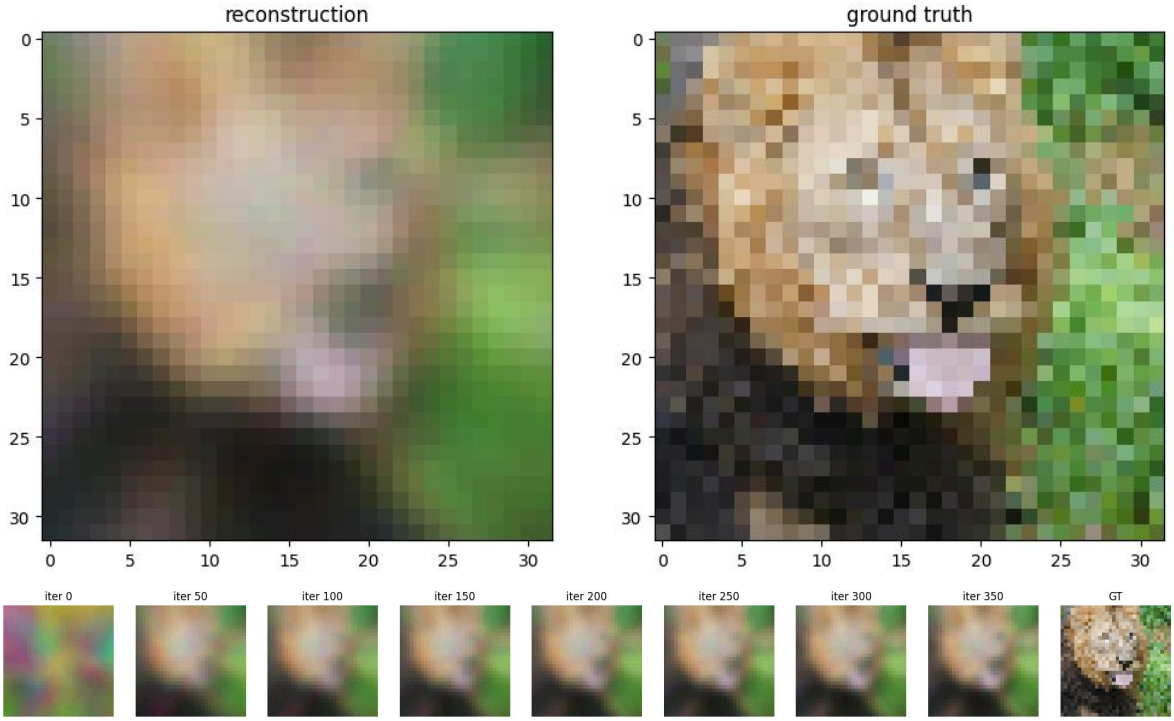
In [12]:
```
# get input features
# TODO implement this by using the get_B_dict() and get_input_features() helper
X_train, y_train, X_test, y_test = get_input_features(B_dict, "none")
opt = "SGD"

# run NN experiment on input features
# TODO implement by using the NN_experiment() helper function
net, train_psnr, test_psnr, train_loss, predicted_images = NN_experiment(X_train
                                                                          X_train
                                                                          output_


# plot results of experiment
plot_training_curves(train_loss, train_psnr, test_psnr)
plot_reconstruction(net.forward(X_test), y_test)
plot_reconstruction_progress(predicted_images, y_test)
```

```
0%|          | 0/400 [00:00<?, ?it/s]
```






```
Final Test MSE 0.012271973296850598
Final Test psnr 16.100556027313615
```



Low Resolution Reconstruction - SGD - Various Input Mapping Stategies

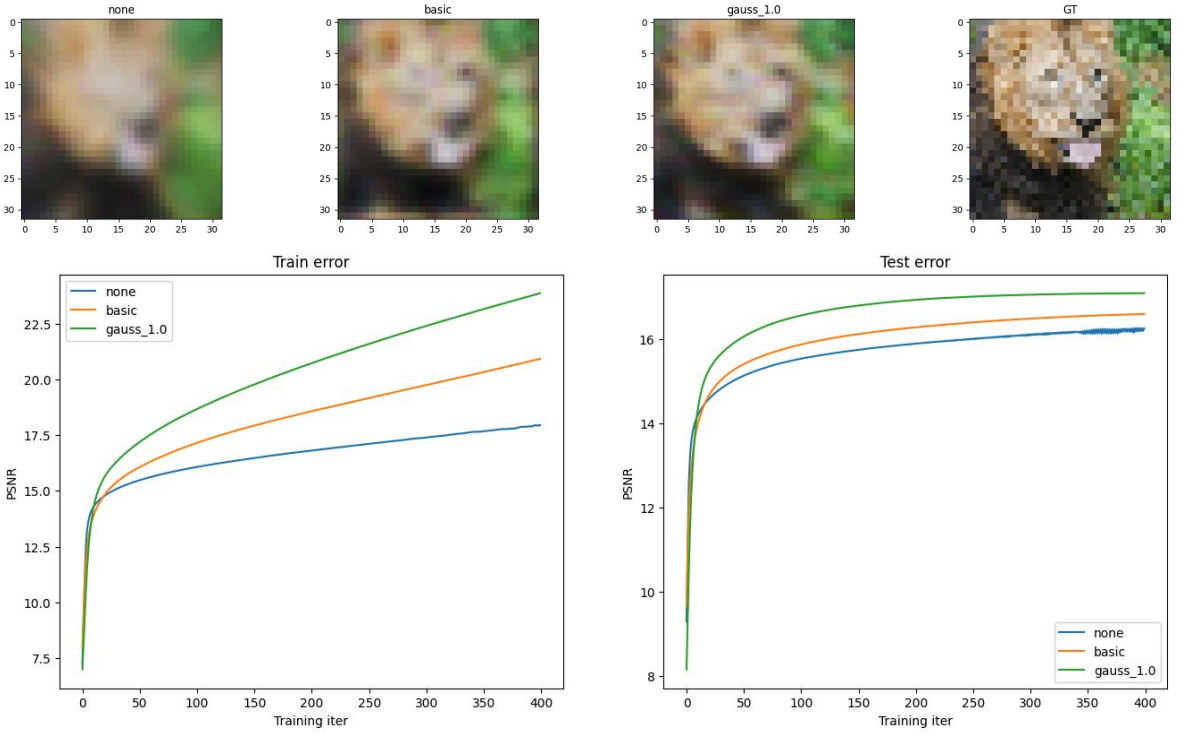
In [13]:
```
def train_wrapper(mapping, size, num_layers, hidden_size, output_size, epochs, u
    # TODO implement me
    # makes it easy to run all your mapping experiments in a for loop
    # this will similar to what you did previously in the last two sections
```

```
    X_train, y_train, X_test, y_test = get_input_features(B_dict, mapping)
    net, train_psnrs, test_psnrs, train_loss, predicted_images = NN_experiment(X
                                                                                X_train
                                                                                output_


    return {
        'net': net,
        'train_psnrs': train_psnrs,
        'test_psnrs': test_psnrs,
        'train_loss': train_loss,
        'pred_imgs': predicted_images
    }
```

In [14]:
```
outputs = {}
for k in tqdm(B_dict):
  print("training", k)
  outputs[k] = train_wrapper(k, size, num_layers, hidden_size_low, output_size,
```

```
  0%|          | 0/3 [00:00<?, ?it/s]
training none
  0%|          | 0/400 [00:00<?, ?it/s]
training basic
  0%|          | 0/400 [00:00<?, ?it/s]
training gauss_1.0
  0%|          | 0/400 [00:00<?, ?it/s]
```

In [15]:
```
# if you did everything correctly so far, this should output a nice figure you c
plot_feature_mapping_comparison(outputs, y_test.reshape(size,size,3))
```



High Resolution Reconstruction

High Resolution Reconstruction - SGD - Various Input Mapping Stategies

Repeat the previous experiment, but at the higher resolution. The reason why we have you first experiment with the lower resolution since it is faster to train and debug. Additionally, you will see how the mapping strategies perform better or worse at the two different input resolutions.

In [16]:
```
# load hi-res image
size = 128
train_data, test_data = get_image(size)
```

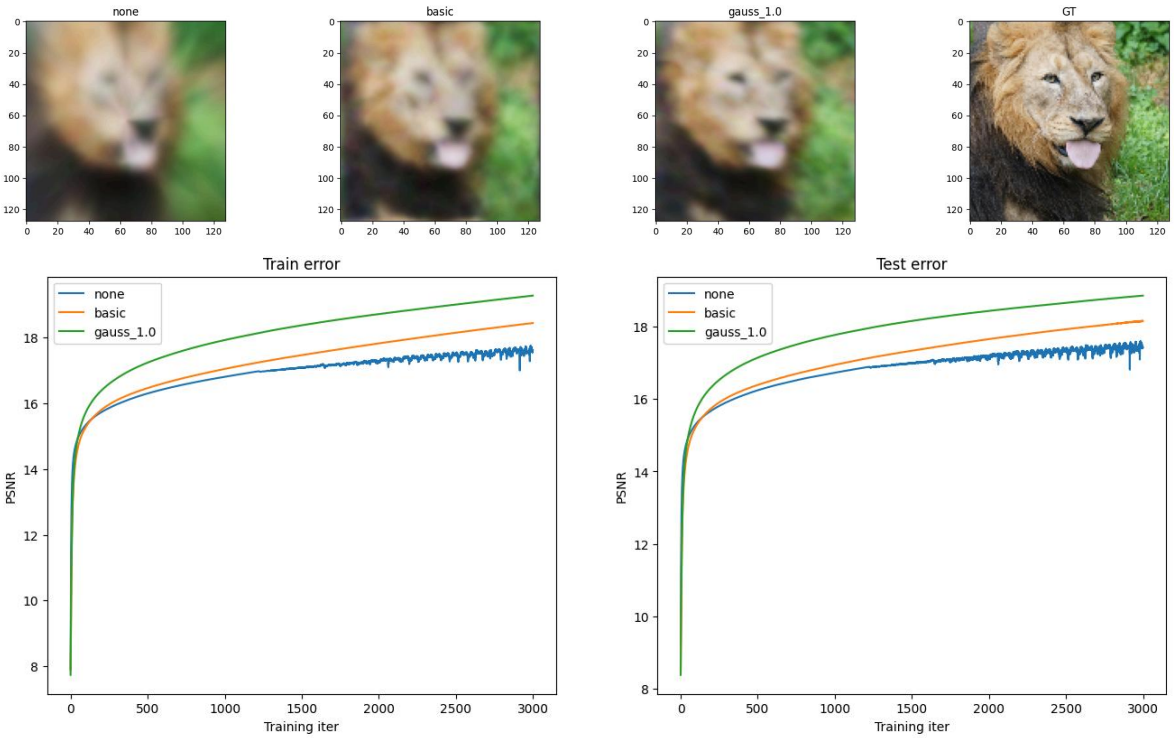
```
C:\Users\wanglibin\AppData\Local\Temp\ipykernel_12200\1559754012.py:6: Deprecatio
nWarning: Starting with ImageIO v3 the behavior of this function will switch to t
hat of iio.v3.imread. To keep the current behavior (and make this warning disappe
ar) use `import imageio.v2 as imageio` or call `imageio.v2.imread` directly.
  img = imageio.imread(image_url)[..., :3] / 255.
```

In [17]:
```
outputs = {}

for k in tqdm(B_dict):
  print("training", k)
  outputs[k] = train_wrapper(k,size, num_layers, hidden_size_high, output_size,
```

```
  0%|          | 0/3 [00:00<?, ?it/s]
training none
  0%|          | 0/3000 [00:00<?, ?it/s]
```

```
training basic
  0%|          | 0/3000 [00:00<?, ?it/s]
training gauss_1.0
  0%|          | 0/3000 [00:00<?, ?it/s]
```

In [18]:
```
X_train, y_train, X_test, y_test = get_input_features(get_B_dict(size), "none")

# if you did everything correctly so far, this should output a nice figure you c
plot_feature_mapping_comparison(outputs, y_test.reshape(size,size,3))
```

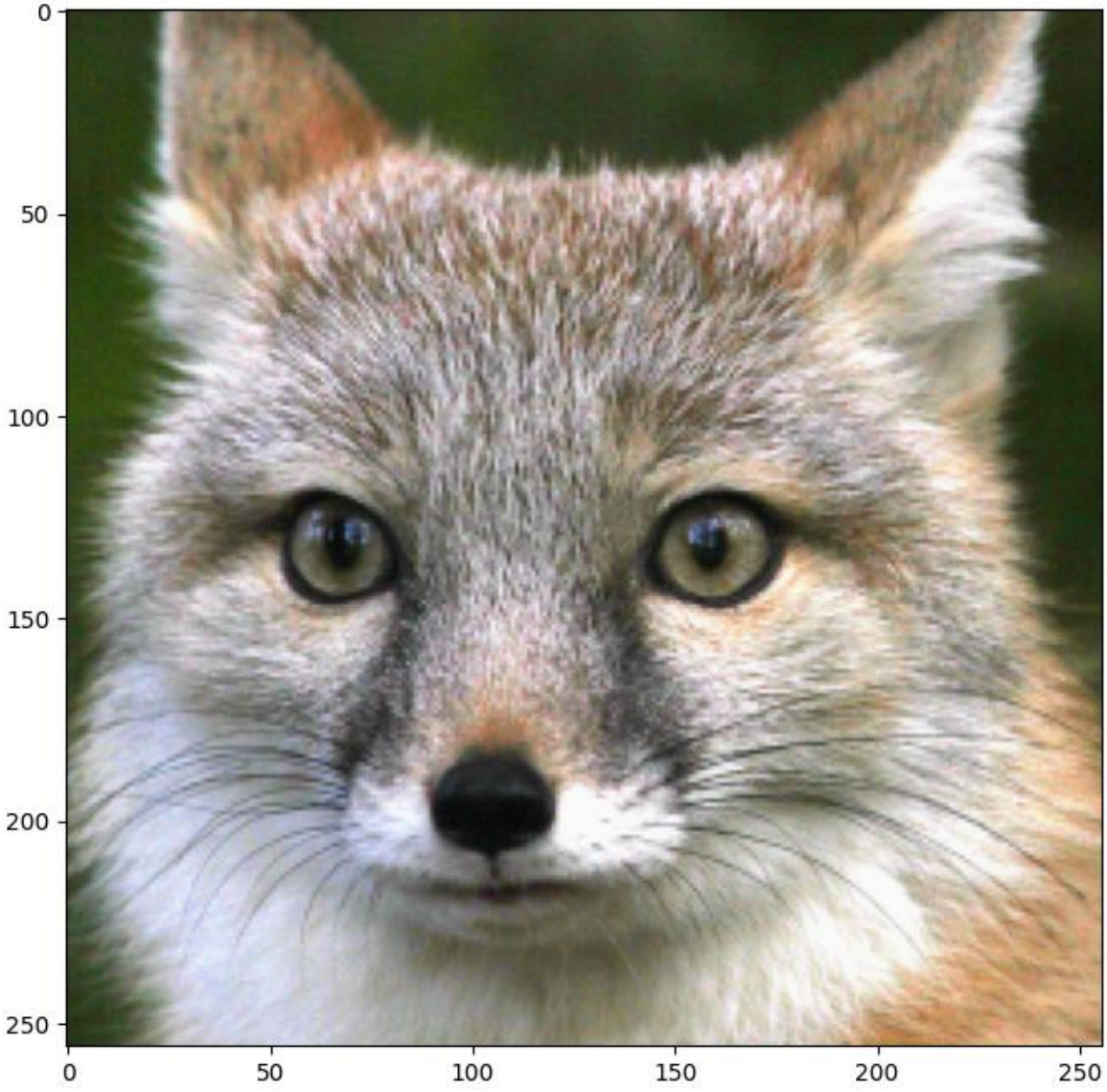


High Resolution Reconstruction - Image of your Choice

When choosing an image select one that you think will give you interesting results or a better insight into the performance of different feature mappings and explain why in your report template.

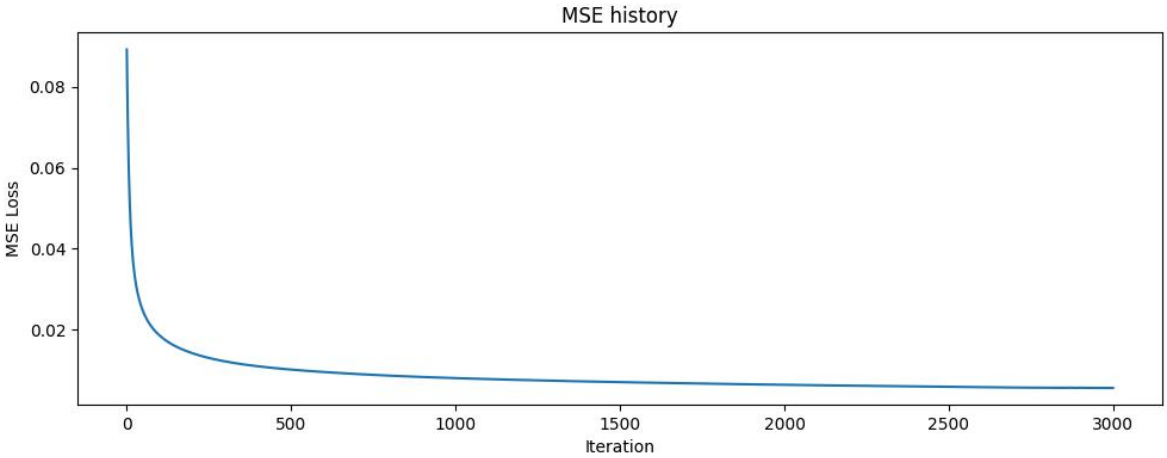
In [19]:
```
size = 256
# TODO pick an image and replace the url string
train_data, test_data = get_image(size, image_url="https://live.staticflickr.com
```

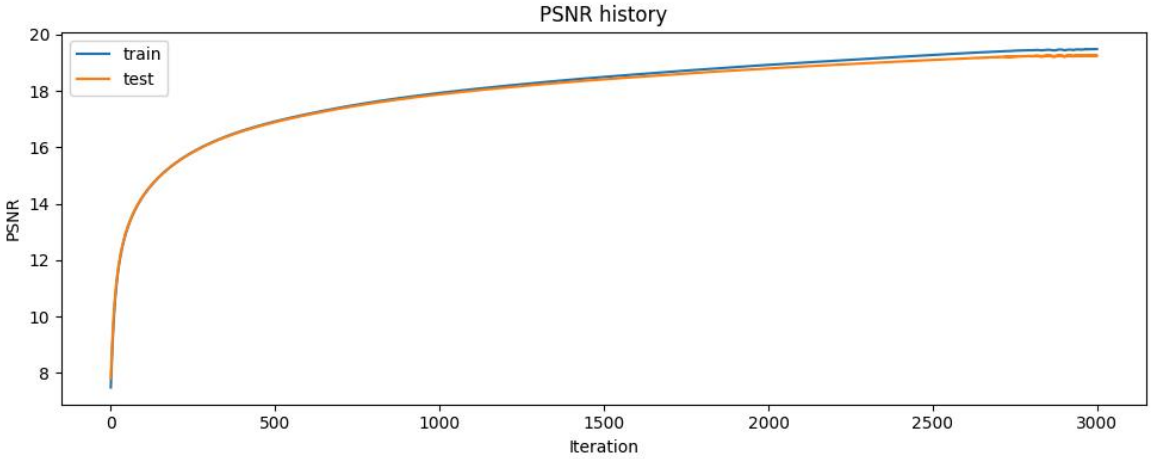
```
C:\Users\wanglibin\AppData\Local\Temp\ipykernel_12200\1559754012.py:6: Deprecatio
nWarning: Starting with ImageIO v3 the behavior of this function will switch to t
hat of iio.v3.imread. To keep the current behavior (and make this warning disappe
ar) use `import imageio.v2 as imageio` or call `imageio.v2.imread` directly.
  img = imageio.imread(image_url)[..., :3] / 255.
```



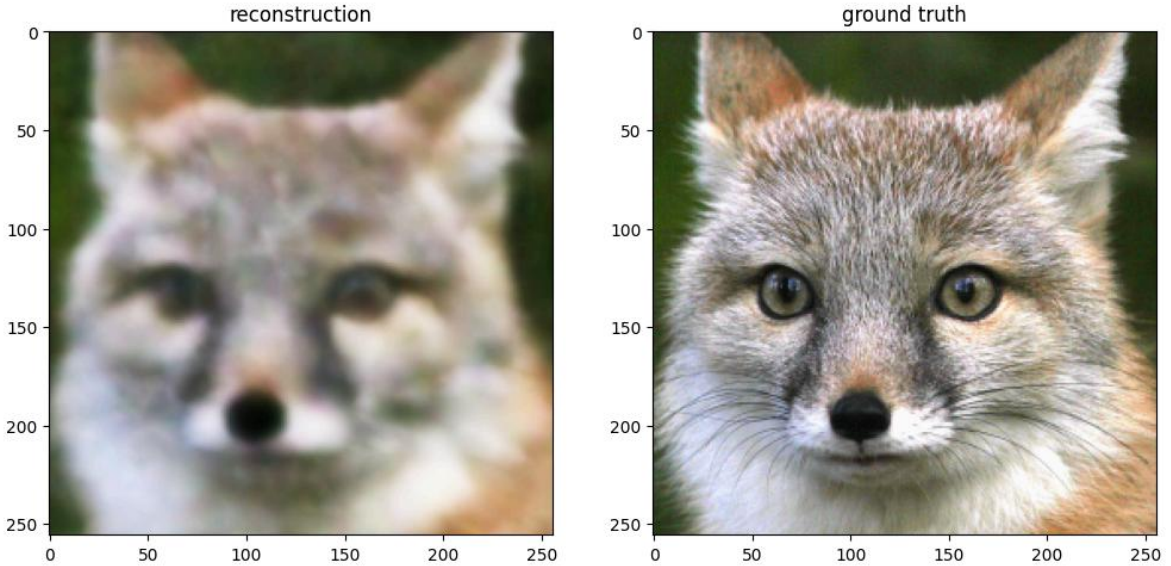
In [21]:
```
# get input features
# TODO implement this by using the get_B_dict() and get_input_features() helper
X_train, y_train, X_test, y_test = get_input_features(get_B_dict(size), "gauss_1
opt = "SGD"

# run NN experiment on input features
# TODO implement by using the NN_experiment() helper function
net, train_psnr, test_psnr, train_loss, predicted_images = NN_experiment(X_train
                                                                         X_train
                                                                         output_


plot_training_curves(train_loss, train_psnr, test_psnr)
plot_reconstruction(net.forward(X_test), y_test)
plot_reconstruction_progress(predicted_images, y_test)
```

```
0%|          | 0/3000 [00:00<?, ?it/s]
```






```
Final Test MSE 0.005941006588994597
Final Test psnr 19.251099702959067
```



Reconstruction Process Video (Optional)

(For Fun!) Visualize the progress of training in a video

In [ ]:
```
# requires installing this additional dependency
# !pip install imageio-ffmpeg
```

In [ ]:
```
# single video example
# create_and_visualize_video({"gauss": {"pred_imgs": predicted_images}}, filenam
```

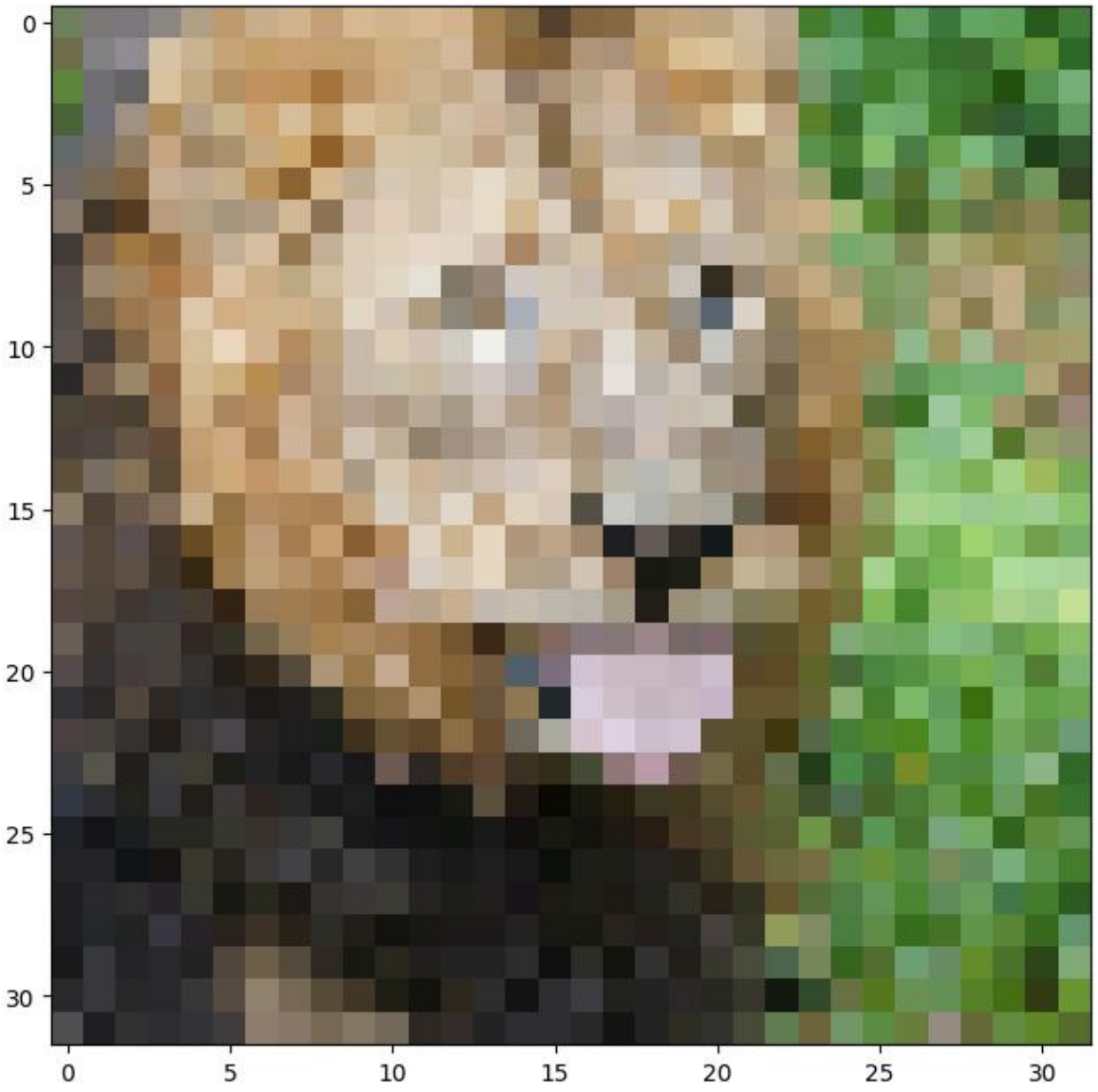
In [ ]:
```
# multi video example
# create_and_visualize_video(outputs, epochs=1000, size=32)
```

Extra Credit - Adam Optimizer

Low Resolution Reconstruction - Adam - None Mapping

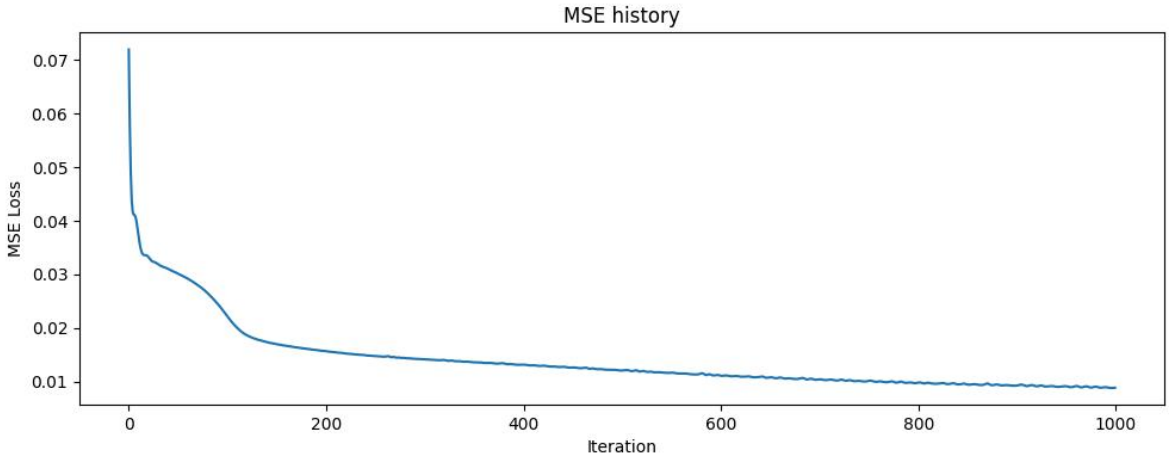
In [22]:
```
# load low-res image
size = 32
train_data, test_data = get_image(size)
```

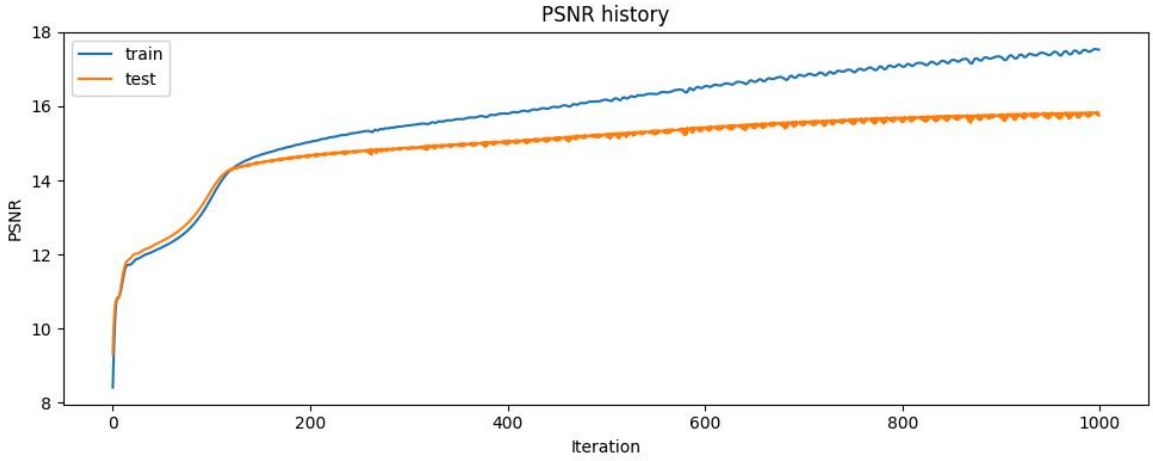
```
C:\Users\wanglibin\AppData\Local\Temp\ipykernel_12200\1559754012.py:6: Deprecatio
nWarning: Starting with ImageIO v3 the behavior of this function will switch to t
hat of iio.v3.imread. To keep the current behavior (and make this warning disappe
ar) use `import imageio.v2 as imageio` or call `imageio.v2.imread` directly.
  img = imageio.imread(image_url)[..., :3] / 255.
```



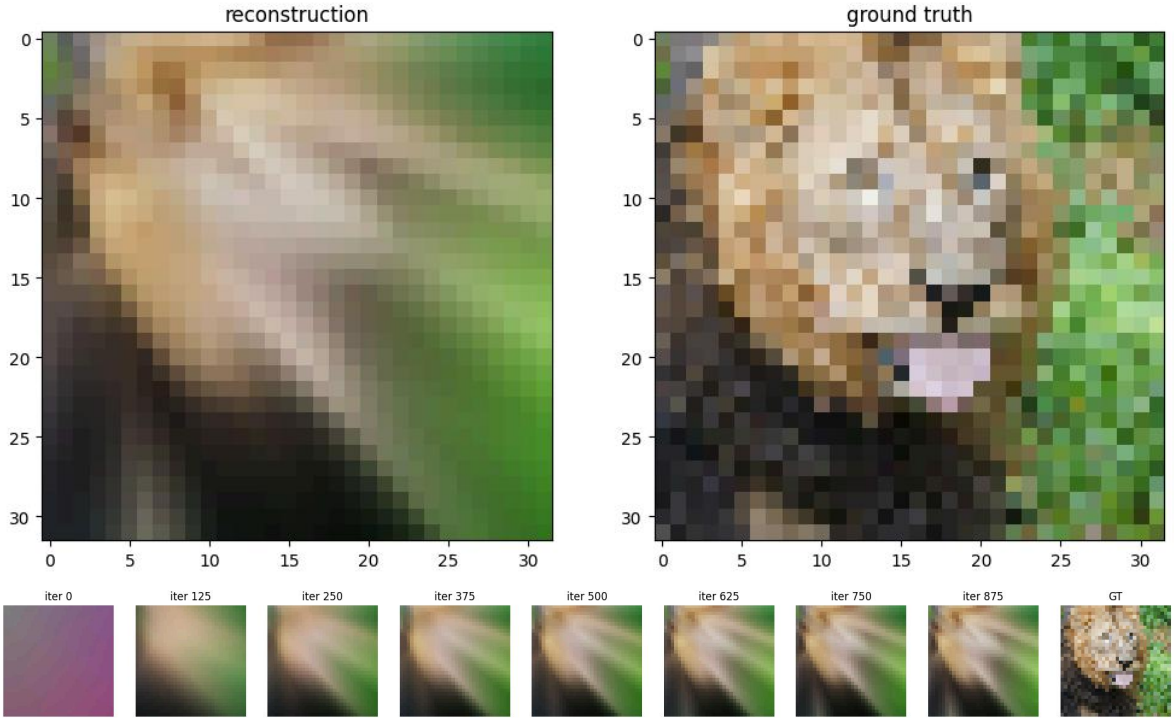
In [23]:
```
# get input features
# TODO implement this by using the get_B_dict() and get_input_features() helper
X_train, y_train, X_test, y_test = get_input_features(get_B_dict(size), "none")
opt = "Adam"
use_bn_adam = False

# set hyperparameters
hidden_size = 256
epochs = 1000
learning_rate = 0.0005

# run NN experiment on input features
# TODO implement by using the NN_experiment() helper function
net, train_psnr, test_psnr, train_loss, predicted_images = NN_experiment(X_train
                                                                          X_train
                                                                          output_


plot_training_curves(train_loss, train_psnr, test_psnr)
plot_reconstruction(net.forward(X_test), y_test)
plot_reconstruction_progress(predicted_images, y_test)
```

```
0%|          | 0/1000 [00:00<?, ?it/s]
```






```
Final Test MSE 0.013296359818421274
Final Test psnr 15.752372451101511
```

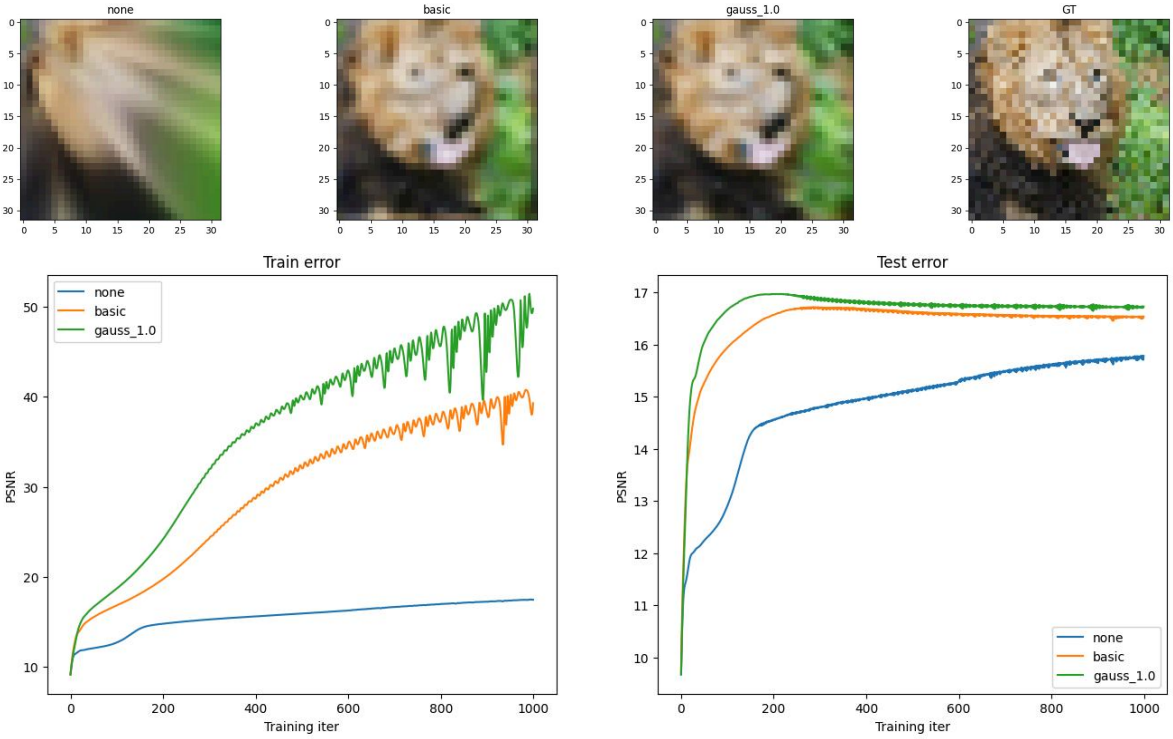


Low Resolution Reconstruction - Adam - Various Input Mapping Stategies

In [24]:
```
# set hyperparameters
hidden_size = 256
epochs = 1000
learning_rate = 0.0003
```

```
# start training
outputs = {}
for k in tqdm(B_dict):
  print("training", k)
  outputs[k] = train_wrapper(k, size, num_layers, hidden_size, output_size, epoc

X_train, y_train, X_test, y_test = get_input_features(get_B_dict(size), "none")

# if you did everything correctly so far, this should output a nice figure you c
plot_feature_mapping_comparison(outputs, y_test.reshape(size,size,3))
```

```
  0%|          | 0/3 [00:00<?, ?it/s]
training none
  0%|          | 0/1000 [00:00<?, ?it/s]
training basic
  0%|          | 0/1000 [00:00<?, ?it/s]
training gauss_1.0
  0%|          | 0/1000 [00:00<?, ?it/s]
```



High Resolution Reconstruction - Adam - Various Input Mapping Stategies

Repeat the previous experiment, but at the higher resolution. The reason why we have you first experiment with the lower resolution since it is faster to train and debug. Additionally, you will see how the mapping strategies perform better or worse at the two different input resolutions.

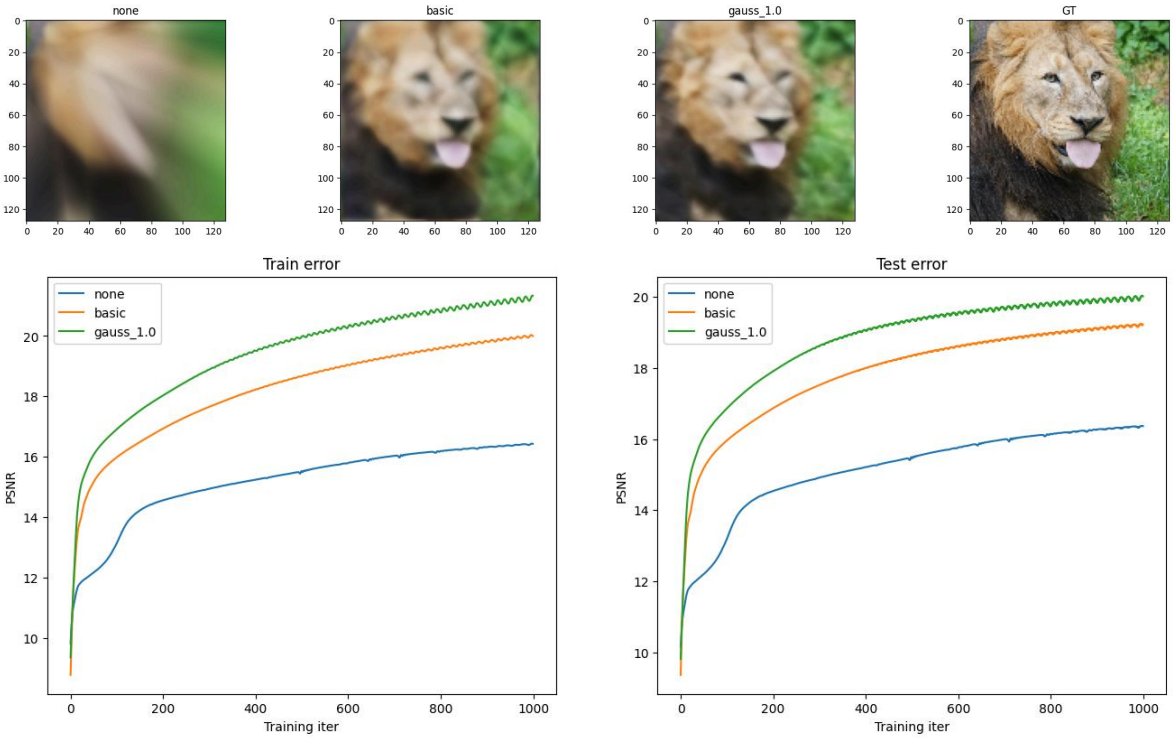
In [25]:
```
# load image
size = 128
train_data, test_data = get_image(size)

# start training
outputs = {}
for k in tqdm(B_dict):
```

```
  print("training", k)
  outputs[k] = train_wrapper(k, size, num_layers, hidden_size, output_size, epoc
```

```
C:\Users\wanglibin\AppData\Local\Temp\ipykernel_12200\1559754012.py:6: Deprecatio
nWarning: Starting with ImageIO v3 the behavior of this function will switch to t
hat of iio.v3.imread. To keep the current behavior (and make this warning disappe
ar) use `import imageio.v2 as imageio` or call `imageio.v2.imread` directly.
  img = imageio.imread(image_url)[..., :3] / 255.
```


```
  0%|          | 0/3 [00:00<?, ?it/s]
training none
  0%|          | 0/1000 [00:00<?, ?it/s]
training basic
  0%|          | 0/1000 [00:00<?, ?it/s]
training gauss_1.0
  0%|          | 0/1000 [00:00<?, ?it/s]
```

In [26]:
```
X_train, y_train, X_test, y_test = get_input_features(get_B_dict(size), "none")

# if you did everything correctly so far, this should output a nice figure you c
plot_feature_mapping_comparison(outputs, y_test.reshape(size,size,3))
```



Extract Credit -- Pytorch Implementation

In [27]:
```
import torch
import torch.nn as nn
from models.neural_net_pytorch import NeuralNet
```

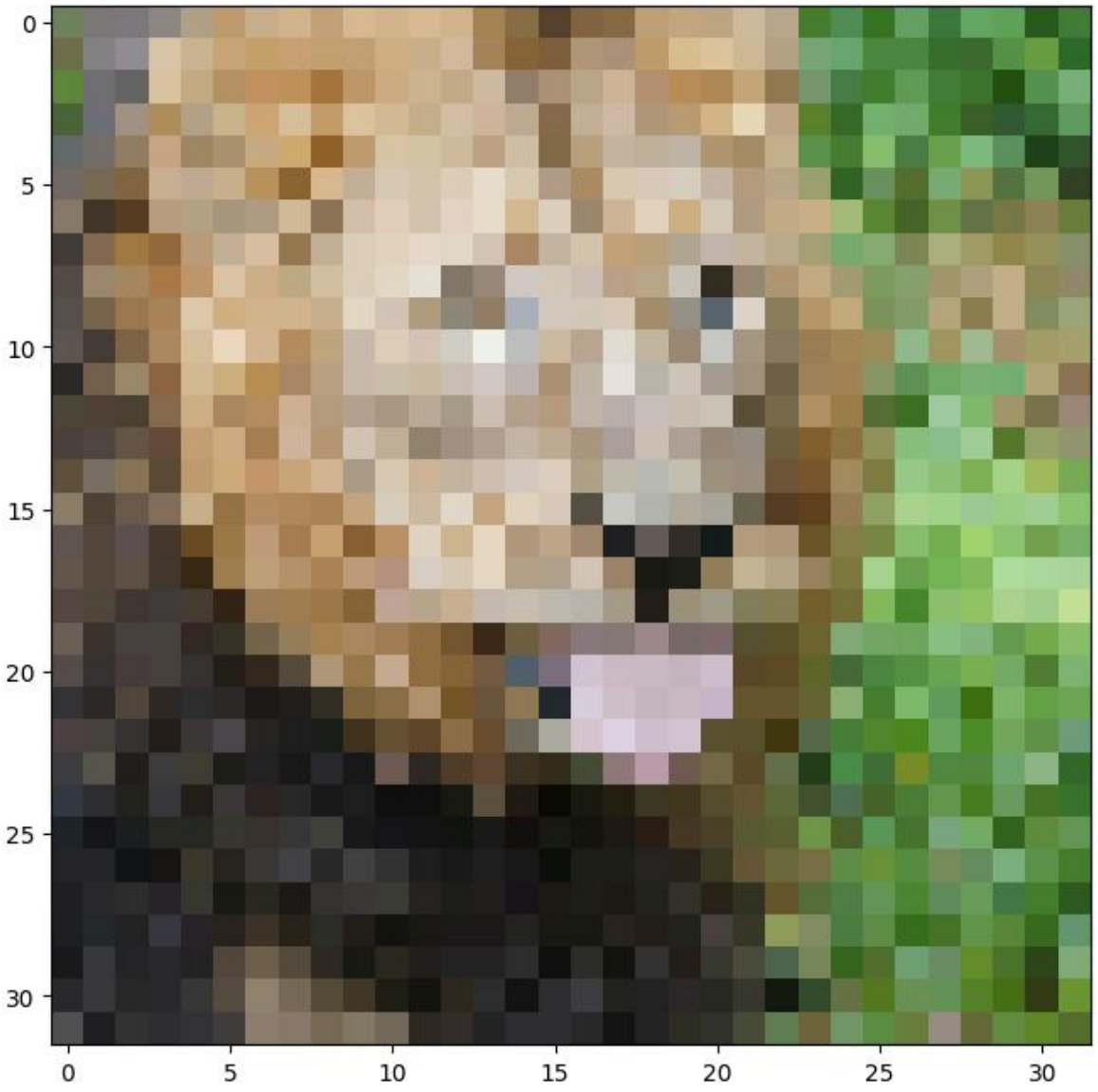
In [28]:
```
def train_nn_experiment(X_train, y_train, X_test, y_test, input_size, num_layers
                        hidden_size, output_size, epochs, use_bn, learning_rate,
    """Train a PyTorch neural network model and evaluate its performance."""

    # Convert NumPy arrays to PyTorch tensors
    X_train = torch.tensor(X_train, dtype=torch.float32)
    y_train = torch.tensor(y_train, dtype=torch.float32)
    X_test = torch.tensor(X_test, dtype=torch.float32)
    y_test = torch.tensor(y_test, dtype=torch.float32)

    # Initialize model
    hidden_sizes = [hidden_size] * (num_layers - 1)
    net = NeuralNet(input_size, hidden_sizes, output_size, num_layers, use_bn, l

    # Variables to store performance for each epoch
    train_loss = np.zeros(epochs)
    train_psnr = np.zeros(epochs)
    test_psnr = np.zeros(epochs)
    predicted_images = np.zeros((epochs, y_test.shape[0], y_test.shape[1]))

    # Training loop
    for epoch in tqdm(range(epochs)):
        # Shuffle dataset
        N = X_train.shape[0]
        indices = torch.randperm(N)
        X_train_shuffled = X_train[indices]
        y_train_shuffled = y_train[indices]
```

```
        # Training step
        loss = net.backward_pass(X_train_shuffled, y_train_shuffled)
        train_loss[epoch] = loss

        # Compute training PSNR
        with torch.no_grad():
            train_pred = net.forward(X_train)
            train_psnr[epoch] = psnr(train_pred.numpy(), y_train.numpy())

        # Compute testing PSNR
        with torch.no_grad():
            test_pred = net.forward(X_test)
            test_psnr[epoch] = psnr(test_pred.numpy(), y_test.numpy())
            predicted_images[epoch] = test_pred.numpy()

        # Update learning rate
        for param_group in net.optimizer.param_groups:
            param_group['lr'] *= lr_decay

    return net, train_psnr, test_psnr, train_loss, predicted_images
```

Low Resolution Reconstruction - SGD - Various Input Mapping Strategies

In [29]:
```
# load low-res image
size = 32
train_data, test_data = get_image(size)
```

```
C:\Users\wanglibin\AppData\Local\Temp\ipykernel_12200\1559754012.py:6: DeprecationWarning: Starting with ImageIO v3 the behavior of this function will switch to that of iio.v3.imread. To keep the current behavior (and make this warning disappear) use `import imageio.v2 as imageio` or call `imageio.v2.imread` directly.
  img = imageio.imread(image_url)[..., :3] / 255.
```

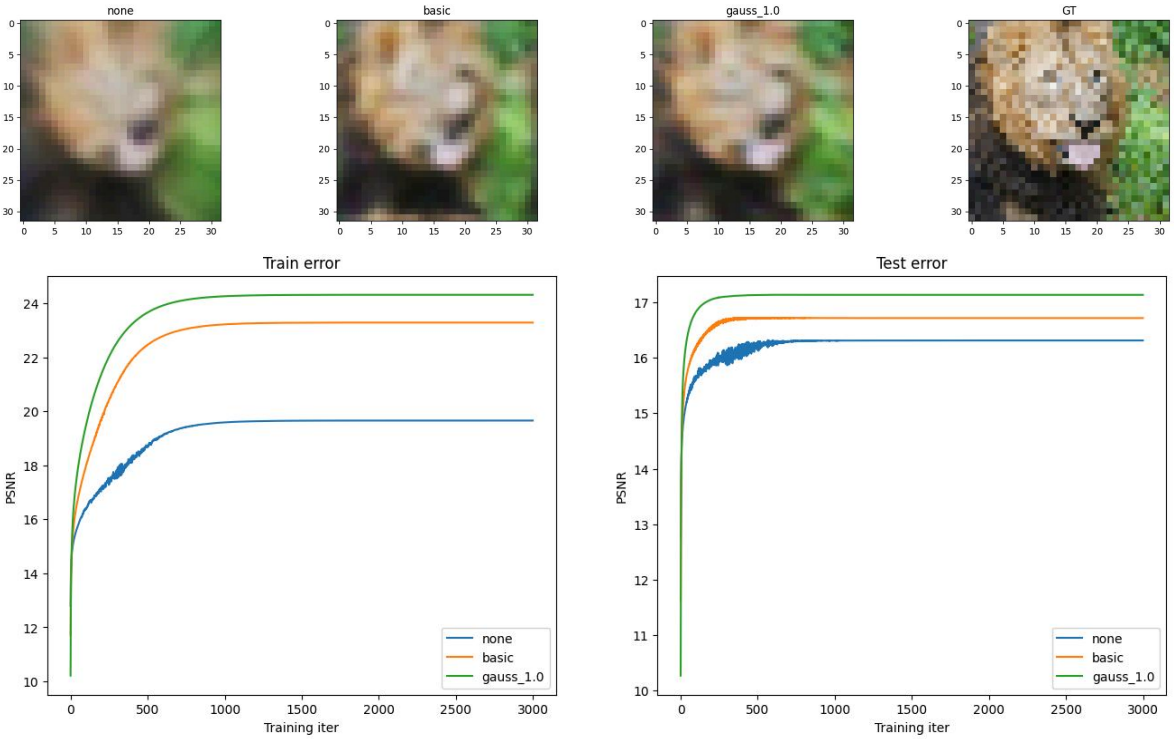


In [30]:
```
def train_wrapper_pytorch(mapping, size, num_layers, hidden_size, output_size, e
    X_train, y_train, X_test, y_test = get_input_features(B_dict, mapping)
    net, train_psnrs, test_psnrs, train_loss, predicted_images = train_nn_experi
                                                                 X_train
                                                                 output_


    return {
        'net': net,
        'train_psnrs': train_psnrs,
        'test_psnrs': test_psnrs,
        'train_loss': train_loss,
        'pred_imgs': predicted_images
    }
```

In [31]:
```
# Set hyperparameters
num_layers_pytorch = 4
hidden_size_pytorch = 256
output_size_pytorch = 3
use_bn_pytorch = True
epochs_pytorch = 3000
learning_rate_pytorch = 0.3
lr_decay_pytorch = 0.995

outputs_pytorch = {}
```

```
for k in tqdm(B_dict):
  print("training", k)
  outputs_pytorch[k] = train_wrapper_pytorch(k, size, num_layers_pytorch, hidden
                                             epochs_pytorch, use_bn_pytorch, lea

X_train, y_train, X_test, y_test = get_input_features(get_B_dict(size), "none")
plot_feature_mapping_comparison(outputs_pytorch, y_test.reshape(size,size,3))
```

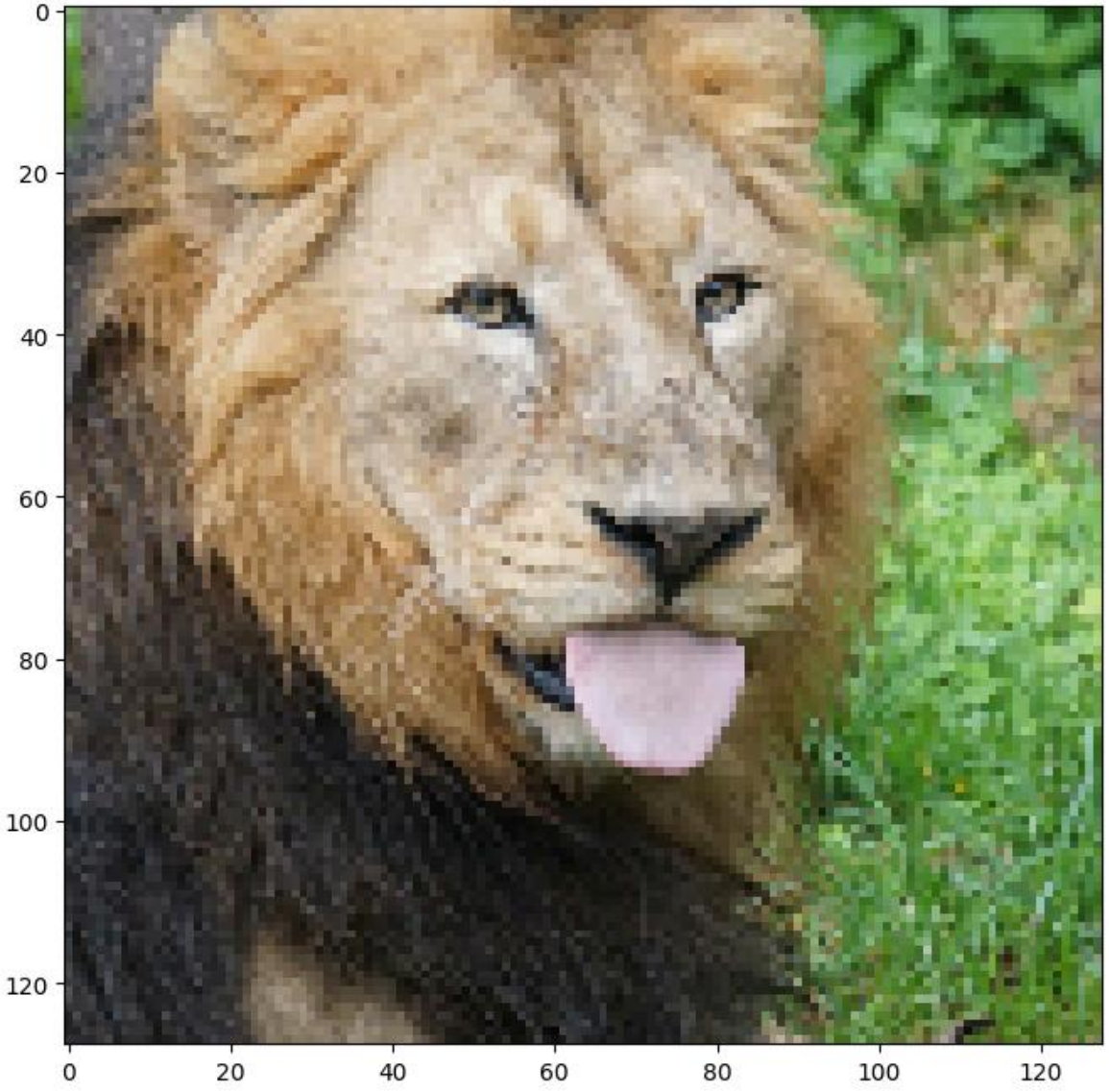
```
  0%|          | 0/3 [00:00<?, ?it/s]
training none
  0%|          | 0/3000 [00:00<?, ?it/s]
training basic
  0%|          | 0/3000 [00:00<?, ?it/s]
training gauss_1.0
  0%|          | 0/3000 [00:00<?, ?it/s]
```



High Resolution Reconstruction - SGD - Various Input Mapping Strategies

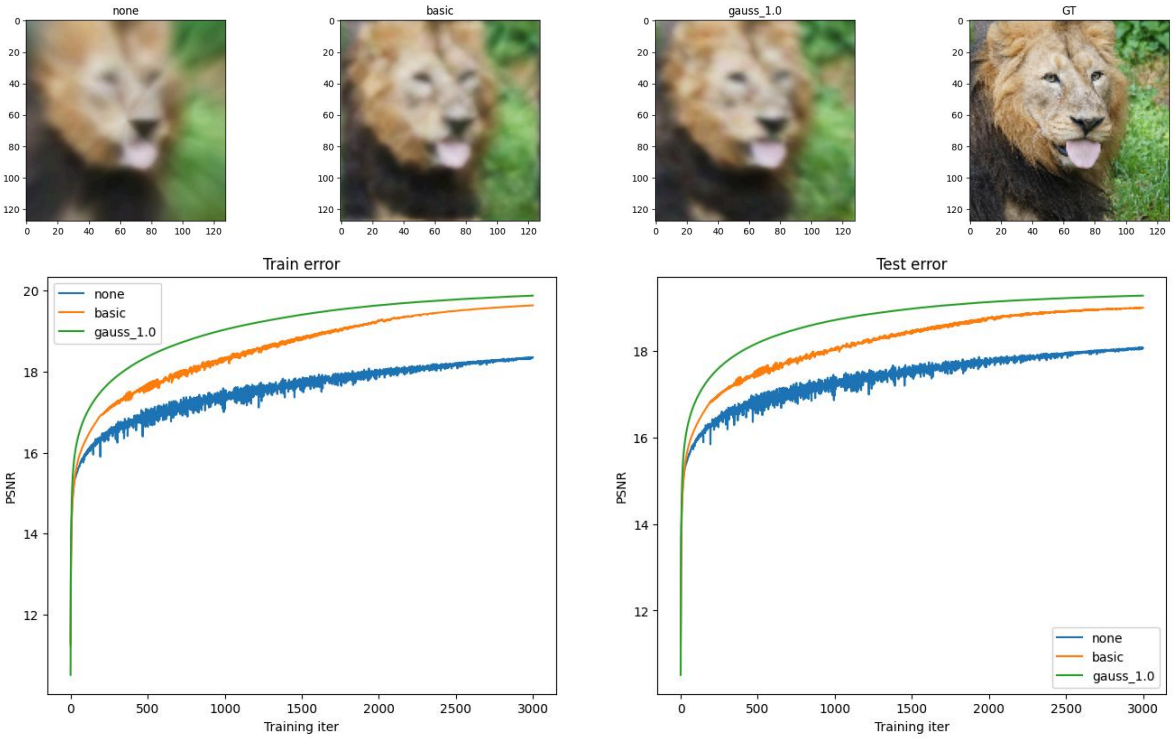
In [32]:
```
# load hi-res image
size = 128
train_data, test_data = get_image(size)
```

```
C:\Users\wanglibin\AppData\Local\Temp\ipykernel_12200\1559754012.py:6: Deprecatio
nWarning: Starting with ImageIO v3 the behavior of this function will switch to t
hat of iio.v3.imread. To keep the current behavior (and make this warning disappe
ar) use `import imageio.v2 as imageio` or call `imageio.v2.imread` directly.
  img = imageio.imread(image_url)[..., :3] / 255.
```



In [33]:
```
# Set hyperparameters
num_layers_pytorch = 4
hidden_size_pytorch = 256
output_size_pytorch = 3
use_bn_pytorch = True
epochs_pytorch = 3000
learning_rate_pytorch = 0.3
lr_decay_pytorch = 0.999

outputs_pytorch = {}
for k in tqdm(B_dict):
  print("training", k)
  outputs_pytorch[k] = train_wrapper_pytorch(k, size, num_layers_pytorch, hidden
                                             epochs_pytorch, use_bn_pytorch, lea

X_train, y_train, X_test, y_test = get_input_features(get_B_dict(size), "none")
plot_feature_mapping_comparison(outputs_pytorch, y_test.reshape(size,size,3))
```

```
  0%|          | 0/3 [00:00<?, ?it/s]
training none
  0%|          | 0/3000 [00:00<?, ?it/s]
training basic
  0%|          | 0/3000 [00:00<?, ?it/s]
training gauss_1.0
  0%|          | 0/3000 [00:00<?, ?it/s]
```


none
basic
gauss_1.0
GT
Train error
Test error
PSNR
Training iter
none
basic
gauss_1.0